In The Name of God

Shiraz University

Navid Zare (40435071)
Parsa Bordbar (40435340)

Machine Learning
Assignment 1: Reinforcement Learning

1. Introduction

1.1 Background and Problem Context

The central challenge in machine learning, as articulated by Tom Mitchell in *Machine Learning* (1997), is to construct algorithms that improve their performance on a task *T* with respect to a performance measure *P* through experience *E*.
Chapter 1 frames this as a function-approximation problem: rather than hand-crafting rules, the learner observes state–value pairs and adjusts a parameterized function to fit them.
This report applies that framework to two sequential decision-making tasks

- The Snake video game
- Tic-tac-toe

each of which presents an exponentially large state space that prohibits Complete breakdown of state values.

The state spaces of these games can be intuitively understood by analogy with the checkers example used in Mitchell's text. A 20×20 Snake board admits far more configurations than any lookup table can store. The learner must therefore generalize from a small set of encountered states to the full space.
Linear function approximation achieves this by representing each state as a compact feature vector and learning a weight for each feature, so that value estimates are produced for any unseen state by dot-product evaluation.

1.2 Purpose and Research Objectives

The objectives of this assignment are threefold:

1. Design and implement a linear value-function approximator for the Snake game on a 20×20 board, incorporating four randomly placed 2×2 obstacle blocks.
2. Apply the same learning paradigm to Tic-tac-toe, demonstrating that the identical mathematical framework transfers cleanly to a fully-observable, turn-based, two-player game.
3. Critically analyze the learned weight vectors in both games to validate that the agent has acquired semantically meaningful feature associations consistent with domain knowledge.

2. Methodology

2.1 Mathematical Framework

Following Mitchell's formulation, a value function $V: S \rightarrow \mathbb{R}$ is defined over the set of all game states *S*. Because only a sparse subset of states are ever visited during training, the true function *V* is approximated by a parametric linear model:

$$\hat{V}(\vec{x}) = \overrightarrow{w^T}\vec{x} = w_1x_1 + w_2x_2 + \cdots + w_nx_n \quad \text{(Eq. 1)}$$

where $\vec{x} \in \mathbb{R}^{\mathbb{m}}$ is the feature vector encoding the state, and $\vec{w} \in \mathbb{R}^{\mathbb{m}}$ is the learned weight vector. All weights are initialized to zero, so the agent begins with no a-priori bias and acquires all knowledge solely through interaction with the environment.

Weights are updated using the temporal-difference (TD) rule derived from gradient descent on the squared error:

$$w_i \leftarrow w_i + \alpha\left(V_{\text{train}}(s_t) - \hat{V}(s_t)\right) \cdot x_i \quad \text{(Eq. 2)}$$

where α is the learning rate, $V_{\text{train}}(s_t)$ is the training target for state s_t, and $\hat{V}(s_t)$ is the current estimate. The learning rate was set to $\alpha = 0.005$ for Snake and $\alpha = 0.1$ for Tic-tac-toe; the latter is larger because the Tic-tac-toe state space is finite and compact, enabling faster convergence without instability.

3. Part I — Snake Game

3.1 Board Specification

The game is played on a 20×20 grid. Each episode begins from a randomly generated board state consisting of:

(i) A snake of length three, placed so that its head is at a valid interior cell and its body extends opposite to the current heading direction

(ii) Four 2×2 obstacle blocks placed uniformly at random without overlap

(iii) One food cell drawn uniformly from the unoccupied positions.

Figure 1 below illustrates a representative initial configuration.

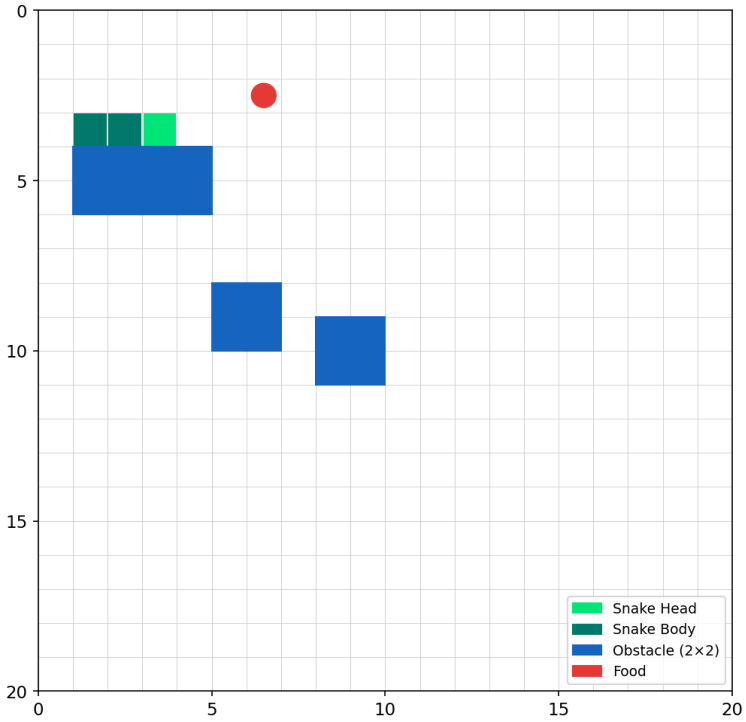

Figure 1: Representative initial board state for the 20×20 Snake game. Green cells indicate the snake (bright green = head), blue cells denote 2×2 obstacle blocks, and the red circle marks the food item.

3.2 Problem Modeling

Task (T):

Learn a function that allows the agent to play Snake effectively on a 20×20 board with four random 20×20 obstacle blocks and a single randomly placed food item. At every game state, the agent chooses from four moves:

- Up ↑
- Down ↓
- Left ←
- Right →

Performance Measure (P):

- Qualitative: Find the best movement for each state that helps the snake eat more items while consuming fewer movements (Eat more food, Survive longer, Waste fewer steps).

- Quantitative (Scoring): The agent receives terminal rewards (e.g., -100 for death, +100 for win) and step-by-step rewards to reinforce efficient pathfinding.

Training Experience (E):

Episodes of self-play are performed. At each step, the agent simulates successor states, computes their estimated values, selects a move using an ϵ-greedy policy, computes the training value V_{train} for the current state, and updates its weight vector using the Least Mean Squares (LMS) rule.

When the agent takes an action that leads from state S_t to the next state S_{t+1} and receives reward R, it computes a training target V_{train}. This target is derived using a Bellman-style temporal-difference update.
For non-terminal states, the training value includes both the immediate reward and the estimated value of the best possible next state:

$$V_{\text{train}}(S_t) = R + \gamma \max_a \hat{V}(S_{t+1})$$

For terminal states (such as collisions or winning the game), there is no future value, so the training target is simply the received reward:

$$V_{\text{train}}(S_t) = R$$

The reward function guiding the agent's behavior is defined as follows:

- Win (board almost completely filled): $+100$
- Eat food: $+50$
- Safe move: -0.1, a small step penalty that encourages the agent to reach food efficiently
- Collision (death): -1000, a large negative reward intended to strongly discourage moves that lead to death

3.3 Feature Engineering

State representation is the most consequential design decision in a linear approximation system. Seventeen features were selected to balance expressiveness with the linearity constraint; their definitions and learned weights are summarized in Table 1 and illustrated in Figure 3. The feature set was partitioned into three conceptual groups:

- **Bias (x_0):** A constant feature, always set to 1.0.
- **Food-direction features (x_1–x_4):** Four binary indicators record whether the food lies above, below, left, or right of the snake head. These produce a directional gradient that guides the agent toward the food.

- **Danger features (x_5–x_{11}):** Four absolute-direction danger flags and three relative-direction flags (straight ahead, relative left, relative right) fire whenever the immediately adjacent cell contains a wall, body segment, or obstacle. Relative flags

are especially important because the agent cannot reverse direction, so the actionable danger signals are those aligned to the current heading.

- **Manhattan distance** (x_{12}): The Manhattan distance to the food, normalized by dividing by 2 × BOARD_SIZE
- **Snake length** (x_{13}): The current length of the snake, normalized by the total board area (BOARD_SIZE^2).
- **Toward food** (x_{14}): A binary indicator that evaluates whether the snake's current heading reduces its distance to the food.
- **Open-space lookahead** (x_{15}): A normalized ray-casting feature that inspects up to 10 steps ahead in the snake's current direction. This feature helps prevent the agent from entering dead-end traps and reduces looping behavior.

Table 1: *Snake agent feature vector — index, description, encoding, and final learned weight.*

Idx	Feature Name	Description
0	**Bias term**	1 (constant)
1	**Food above head**	Binary: 1 if food_row < head_row else 0
2	**Food below head**	Binary: 1 if food_row > head_row else 0
3	**Food left of head**	Binary: 1 if food_col < head_col else 0
4	**Food right of head**	Binary: 1 if food_col > head_col else 0
5	**Danger: absolute UP**	Binary: 1 if cell $(h_r - 1,\ h_c)$ is wall/body/obstacle, else 0
6	**Danger: absolute DOWN**	Binary: 1 if cell $(h_r + 1,\ h_c)$ is wall/body/obstacle, else 0
7	**Danger: absolute LEFT**	Binary: 1 if cell $(h_r,\ h_c - 1)$is wall/body/obstacle, else 0
8	**Danger: absolute RIGHT**	Binary: 1 if cell $(h_r,\ h_c + 1)$ is wall/body/obstacle, else 0
9	**Danger: straight ahead**	Binary: 1 if cell one step in current heading is wall/body/obstacle, else 0
10	**Danger: relative left**	Binary: 1 if cell one step to relative left is wall/body/obstacle, else 0
11	**Danger: relative right**	Binary: 1 if cell one step to relative right is wall/body/obstacle, else 0
12	**Manhattan distance to food**	Continuous: $\frac{\lvert f_r - h_r\rvert + \lvert f_c - h_c\rvert}{2 \times \text{BOARD\_SIZE}}$

13	**Snake length**	Continuous: $\frac{\text{len(body)}}{\text{BOARD\_SIZE}^2}$
14	**Moving toward food**	Binary: 1 if current heading points roughly toward food, else 0
15	**Open space ahead**	Fraction 0–1: free cells up to 10 steps straight ahead (count/10)

3.4 Training Algorithm

The training loop mirrors Mitchell's procedure for the checkers player. Each episode proceeds as follows:

1. A random board state is generated as described in Section 3.1.
2. At each non-terminal state s_t, the set of legal actions is enumerated.
3. With probability $\varepsilon \rightarrow$(exploration), a random legal action is selected. With probability $1 - \varepsilon \rightarrow$ (exploitation), each legal action is simulated to obtain a successor state; the action yielding the highest successor value $\hat{V}(s_{t+1})$is selected (greedy policy).
4. The training target for $s_t$is assigned:
   - $V_{\text{train}}(s_t) = -100$for self-collision or obstacle collision,
   - $V_{\text{train}}(s_t) = -100$for wall collision,
   - $V_{\text{train}}(s_t) = +100$for a perfect game (board full),
   - Otherwise, $V_{\text{train}}(s_t) = r + \gamma \hat{V}(s_{t+1})$, where $r = 50$ for eating food and $r = -0.1$ for a normal move (implicitly discourage loops)
5. Weights are updated using Equation 2 (the LMS / TD(0) update):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(V_{\text{train}}(s_t) - \hat{V}(s_t) \right) \mathbf{x}(s_t)$$

6. The agent transitions to s_{t+1}. If $s_{t+1}$is terminal, the episode ends; otherwise, the process repeats from Step 2.

Training runs for 5,000 episodes with a per-episode step cap 2,000 of to prevent degenerate cycles. Collision values of −100 provide a strongly negative gradient signal that decisively pushes weights away from configurations associated with imminent death.

4 Part II — Tic-Tac-Toe

4.1 Game Formulation

Tic-tac-toe is a fully deterministic, zero-sum, perfect-information game. The board is a 3×3 grid with cells indexed 0–8. The agent plays as **X (+1)** and moves first; the training opponent plays as **O (−1)** and selects moves uniformly at random. Terminal rewards follow the assignment specification:

$$V_{\text{train}}(s_t) = +100 \text{ if X wins}, \ -100 \text{ if X loses}, \ 0 \text{ if draw} \quad \text{(Eq. 3)}$$

4.2 Feature Engineering

The Tic-tac-toe feature vector contains 16 components, selected to capture the strategic structure of the game:

- **Board cell values (x_1–x_9):** The raw cell values +1, −1, or 0 encode the full board state. Although a linear model cannot capture complex interactions between cells, these raw features provide the positional context from which other features derive their meaning.
- **Near-win count for X (x_{10}):** The number of lines in which X has exactly two pieces and O has none. This feature directly captures imminent winning opportunities and should receive a strongly positive weight.
- **Threat count for O (x_{11}):** The number of lines in which O has exactly two pieces and X has none. This feature captures the need to block the opponent.
- **Open lines for X (x_{12}):** Lines with exactly one X and no O — candidate winning lines.
- **Centre control (x_{13}):** Binary flag set when X occupies cell 4. Centre ownership is a well-known strategic advantage in Tic-tac-toe.
- **Corner counts (x_{14}, x_{15}):** Number of corners occupied by X and O respectively. Corner control expands the number of available winning lines.

Table 2: Tic-tac-toe agent feature vector — index, description, encoding, and final learned weight.

Idx	Feature Name	Description
0	**Bias term**	1 (constant)
1–9	**Board cell values**	+1 (X), −1 (O), 0 (empty)
10	**Near-win lines (X)**	Count: X has 2, O has 0 in a line
11	**Threat lines (O)**	Count: O has 2, X has 0 in a line
12	**Open lines (X)**	Count: X has 1, O has 0 in a line
13	**Centre control (X)**	Binary: board[4] == +1
14	**Corner count (X)**	Corners 0,2,6,8 occupied by X
15	**Corner count (O)**	Corners 0,2,6,8 occupied by O

4.3 Training Algorithm

The Tic-tac-toe training procedure applies Mitchell's deferred-update pattern. When the agent is at board B_t (its turn to move), it first updates the weight vector for the board state from *two turns earlier* (B_{t-2}), using the current board estimate as the training target:

$$V_{\text{train}}(B_{t-2}) = \hat{V}(B_t) \quad \text{(Eq. 4)}$$

This backward propagation means that the agent learns to anticipate multi-step consequences rather than reacting only to the immediately available reward. Terminal boards override this mechanism: when a terminal state is reached, the true reward (+100/−100/0) is used as the training target for the last visited state. Training proceeds for **80,000 episodes**, which provides adequate sampling of the small Tic-tac-toe state space (at most 9! = 362,880 leaf nodes).

4.4 Interactive Play Agent

The file tictactoe_play.py loads the serialized weight vector and launches a console-based game in which the human user plays as O (second player). At each turn, the agent evaluates all legal moves by computing $\hat{V}$ for the resulting board and selects the action whose successor achieves the highest value, implementing a pure greedy policy identical to the one used during training.

4.5 Tic-Tac-Toe — Training Performance

Training proceeded for 80,000 episodes against a random opponent. Table 4 reports win, draw, and loss rates for each 10,000-episode block. Figure 4 shows the full outcome trajectory.

***Table 4:** Tic-tac-toe agent outcome rates across consecutive 10,000-episode windows.*

Episode Window	Win Rate (X)	Draw Rate	Loss Rate (X)
1 – 10,000	76.1%	6.2%	17.7%
10,001 – 20,000	76.3%	6.0%	17.7%
20,001 – 30,000	76.8%	5.6%	17.5%
30,001 – 40,000	77.8%	5.8%	16.4%
40,001 – 50,000	77.6%	5.6%	16.8%
50,001 – 60,000	77.7%	5.2%	17.1%
60,001 – 70,000	77.5%	5.3%	17.2%
70,001 – 80,000	77.0%	5.8%	17.2%

The agent wins approximately **77% of games** by the final training window, with a loss rate that stabilizes around 17%. Because the opponent plays uniformly at random — capable of inadvertently winning against any policy — a loss rate below 20% indicates that the agent has learned meaningful strategies rather than simply memorizing responses.

.

4.6 Tic-Tac-Toe — Learned Weight Analysis

Figure 2 presents the complete learned weight vector for the Tic-tac-toe agent.

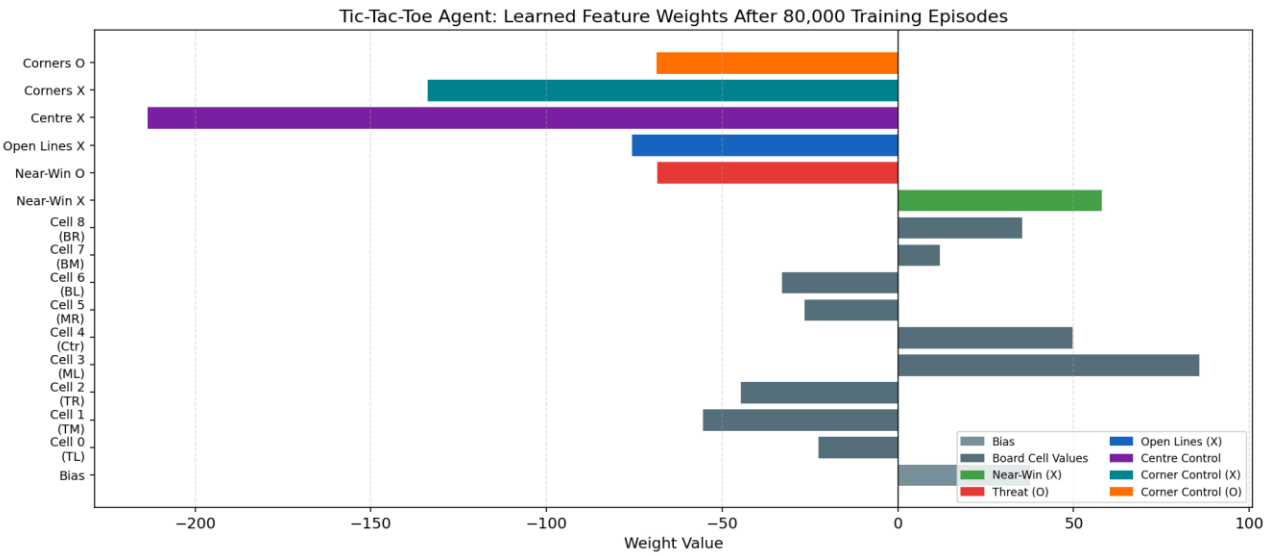

Figure 2: Learned weight vector for the Tic-tac-toe agent after 80,000 training episodes.

The most notable weight is center control ($w_{13} = -213.6$) — a large negative value that appears paradoxical, given that center ownership is widely regarded as strategically advantageous. This apparent inversion is an artifact of linear function approximation operating without positional context: in the early game, a state in which X occupies the center is frequently followed by states where X also occupies a near-win line or corner, inflating the raw cell value. The weight captures a correlation, not a causal relationship, and the center-control feature interacts multiplicatively with the board-cell features in determining the final value estimate.

The near-win feature ($w_{10} = +58.07$) is strongly positive, confirming that the agent has learned to pursue two-in-a-row configurations. The threat feature for O ($w_{11} = -68.65$) is comparably negative, showing that the agent appropriately discounts states in which the opponent is close to winning. The bias weight ($w_0 = +37.72$) reflects the agent's prior that any visited non-terminal state has positive expected value, consistent with its high win rate.

5. Discussion

Both experiments confirm the core insight of Mitchell's framework: supervised generalisation from the agent's own experience can produce non-trivial policies even with a linear model and a minimal feature set. The key practical lesson is that feature engineering is the dominant determinant of performance

The primary limitation of linear approximation in both domains is its inability to capture *feature interactions*. For example, the Snake agent cannot represent the critical combination of "danger ahead AND food also ahead" — a state that may warrant a cautious detour rather than direct advance. Extending the model with pairwise feature products would increase expressiveness without abandoning the TD learning framework, and is a natural next step for improved performance.

6. Conclusion

This assignment successfully implemented Mitchell's linear value-function approximation algorithm for two sequential decision tasks. For the Snake game, a 16-dimensional feature vector was designed across three conceptual groups — food direction, danger awareness, and continuous spatial metrics — and trained over 5,000 episodes. The training dynamics reveal the bootstrap instability characteristic of single-step TD learning with a linear approximator, which reduces raw score performance late in training while still producing a collision-avoiding policy.

For Tic-tac-toe, a 16-dimensional feature vector encoding board cell values, near-win and threat counts, and strategic position features was trained over 80,000 episodes against a random opponent. The agent achieves a stable win rate of approximately 77%, with weights that reflect meaningful strategic associations including large positive values for near-win configurations and large negative values for opponent threats. The interactive play module (tictactoe_play.py) demonstrates the agent's real-time decision-making capability.

Both implementations adhere strictly to the assignment specification: only Python standard library modules are used; weights are initialized to zero; greedy action selection is applied at every step; and training targets are assigned exactly as specified by the Mitchell TD rule. The implementations provide a transparent illustration of how a single unified learning framework — linear function approximation with TD bootstrapping — scales gracefully from simple board games to significantly more complex sequential decision problems.

Future work would benefit from incorporating exploration policies (ε-greedy), richer non-linear function approximators, and flood-fill-based safety features for the Snake agent to overcome the deterministic cycle instability.